

CS – 432: Databases

Assignment – III

Module – A: ACID Properties and B+ Trees

The GitHub Link to access the project content is given below:

https://github.com/Pallav-Biyala/CS432_Track1_Assignment_3_Submission

The link to the YouTube video containing explanation of B+ Tree is:

https://youtu.be/JA4c2rqW_PQ

🌀 Introduction to the Assignment:

In our previous assignment, we saw how to implement B+ Trees and all its operations. We also saw that how B+ Tree is lot better as compared to the Brute Force DB in terms of insertion, deletion, update, range query as well as search. We also saw the graphical representation of B+ Tree through graphviz and showed insertion and deletion of many nodes.

Now, in this assignment, we upgrade our B+ Tree implementation into a **mini-database engine** by introducing a **transactional layer**. The objective is to ensure reliable and consistent database behavior, i.e., it satisfies all ACID properties in the presence of system failures or concurrent user operations. To achieve this, we implement several mechanisms that enforce the **ACID properties** of database systems.

The primary mechanism that we implemented to achieve ACID properties include:

- **Write-Ahead Logging (WAL):** This stores all the logs of the transactions that we performed. It is used because in case a system crashes or transaction fails, then with help of WAL we can recover the database as well as perform rollback.
- **Transaction Management:** It handles transaction lifecycle operations such as **Begin, Commit and Rollback**, ensuring atomic execution of database operations.
- **Check pointing:** It periodically stores the database state to persistent storage to reduce recovery time and balance memory usage with disk persistence. So this is used so that the log file doesn't become too huge and in case of system crash, the database can be recovered quickly.

To evaluate the robustness of the system, a series of tests particular to ACID properties are performed. These experiments demonstrate the ability of the database engine to **recover from failures, maintain data integrity, and handle multi-user conflicts without compromising correctness or consistency**.

So the four properties that we are going to perform are:

1. **Atomicity ("All or Nothing"):** Every operation in a transaction must succeed, or the entire transaction is rolled back. In our engine, we perform this by saving the transactions to wal.log and then to rollback, we extracted the operation from wal.log file and then with help of undo, we performed the inverted operation to get old data back.
2. **Consistency ("Rule Follower"):** The database must transition from one valid state to another. We enforce this through **Primary Key Uniqueness and Entry Validation**.
3. **Isolation ("Stealth Mode"):** Concurrent transactions should not see each other's partial work. We implement this by modifying the B+ Tree only when the transaction is committed.
4. **Durability ("The Survivor"):** Once a transaction is committed, it remains saved even if the power goes out. We ensure this by writing to the **WAL file on disk** before the transaction finishes and also with the help of check points, we maintain the database to be stored in the file after certain number of thresholds.

So now let's move to first topic of our report: Correctness of operations:

☞ Correctness of Operations:

So now we implemented the B+ Tree mini database 2.O. But how do we know its correct or not? This is justified by the correctness of operations!

Correctness of Operations in a database ensures that every transaction produces valid and consistent results while preserving the data integrity of the stored data. In our mini database engine, correctness is ensured through various specific layers. Let's talk about each layer:

→ Structural Correctness (B+ Tree):

So B+ Tree serves as the primary indexing structure of our engine that stores and retrieves records efficiently and in very small amount of time as its time complexity is $O(\log n)$ for most operations. Correctness of operations is ensured by maintaining the fundamental properties of the B+ Tree during insertions, deletions, and updates:

- All leaf nodes remain at the same depth, ensuring balanced tree structure. This makes the graph balanced as well makes it easy to store in the disk.
- Keys within nodes are maintained in **sorted order**, enabling efficient searching. Due to this sorting, we can use the Binary Search properties to find keys. This ensures the $O(\log n)$, very fast retrieval.
- Node splits and merges are performed whenever capacity constraints are violated. This is done so that tree don't become too dense or too sparse. Otherwise operations may take more time
- Internal nodes only store keys for navigation, while actual data records are stored in leaf nodes. This makes it optimal because even if a key is deleted, internal nodes may or may not update (as it updates only if BS property is violated). This increases the speed of deletion.

These properties ensure that search, insertion, deletion, and range query operations always return accurate results while maintaining logarithmic time complexity. Hence, by this way our B+ Tree works efficiently.

→ Logical Correctness:

The database engine is built using three layers:

DB Manager → Table → B+ Tree

The major logical correctness we built was mainly on two aspects:

1. **No Duplicate Primary Key:** Though B+ Tree we built in previous assignment allows duplicate keys, but since B+ Tree uses keys as primary key of a table, it became important for us to now allow it to allow duplicate. This was handled by our table.py.

We made sure that whenever an entry is added to table, before inserting to B+ Tree, we check whether its keys is present or not through B+ Tree's search. If we found that it exists, then we raised Value Error that Duplicate Primary Key is there. And then with help of try and except block, if error raised then we implemented rollback in the except block so that atomicity is maintained.

By this way our engine handles the Duplicate Primary Key.

```
# Inserting row into table
def insert(self, row, tm = None, tid = None):
    key = row[0] # First column as primary key
    if self.index.search(key):
        raise ValueError("Duplicate primary key")
    # Log operation
    if tm and tid:
        op = Operations(self, "insert", key, None, row)
        tm.log_operation(tid, op)
    else:
        self.index.insert(key, row)
    return
```

We can see how we raise ValueError here

2. **No Wrong Updates:** For update operation, we specifically made sure that it doesn't update a value that doesn't exist. This again we used same idea that we search the key if it exists or not in our B+ Tree. If it exists, no problem. Otherwise, we raised ValueError that entry doesn't exist and then with help of try-except block, in except block we added rollback so that atomicity is maintained.

```
# Update Table
def update(self, key, updates, tm=None, tid=None):
    row = self.index.search(key)

    if not row:
        raise ValueError("Entry Doesn't exist")

    new_row = list(row)
```

Wrong Updates: ValueError raised

Hence, by this way we handled logical constraints.

Note: The table.py we created is a very generic table and same template was used for all three tables. So adding specific constraints like cpi>0, roll number format followed etc. was difficult to add as that was specific to each table and we wanted to keep it generic.

Hence, by this way we implemented logical constraints which make sure of Atomicity and Consistency.

→ Transactional Execution:

The backbone of our Upgrade is the **Transaction**. This is one of the main things that made a simple B+ Tree into a powerful mini database engine.

We know, all database operations are executed within transactions. Each transaction follows has a lifecycle:

Begin → Commit / Rollback

So in our engine too we implemented this to ensure that all ACID properties are satisfied.

1. **Begin:** It marks the start of transaction
2. **Commit:** Once a transaction is formed, we need to implement it to our database. Hence, by commit whenever we use it, the B+ Tree performs all the operations: insert, delete, update, search etc. and modifies the database. Until commit is called, B+ Tree can't modify our database.
3. **Rollback:** How can a database be powerful if it doesn't have power to revert its abilities? Many times transactions fail and it becomes necessary that to maintain Atomicity, we need to undo all our operations. So this we did with help of rollback. The rollback knows that what previous operations were in transaction and then by calling operations.py module, it uses it undo operations and then:
 - If operation is delete → it does insert
 - If operation is insert → It does delete
 - If operation is update → It updates the new value to the old value

This is known as the **Undo Logic**, where the WAL provides the 'Before-Image' of the data to revert the database to its state prior to the failed transaction.

```
def undo(self):
    if self.op_type == "update":
        # we updated the key to new value. since we need to rollback, we need to get the old value back
        self.table.index.update(self.key, self.old_value)

    elif self.op_type == "insert":
        # Since we inserted key, in rollback need to delete it
        self.table.index.delete(self.key)

    elif self.op_type == "delete":
        # Since we deleted key, need to insert the key again and its old value
        self.table.index.insert(self.key, self.old_value)

    # Hence, by this way we can rollback
```

By this way, it undoes the failed transaction and thereby maintaining atomicity. Hence, rollback is very important part of our engine

Hence, this ensures that partial or inconsistent updates do not remain in the system which is one of the most important property of any database engine.

→ Write Ahead Logging (WAL):

We know, we want to rollback but how to rollback? So for this we have **WAL**.

WAL means as we call commit, then before modifying the database using B+ Tree, the operation in transaction must be written to the logs file: wal.log. Hence, WAL ensures that all transactions performed are stored in the logs file. At time of rollback, transaction_manager module uses the transactions in WAL and then undo them.

Hence, if transactions are backbone of the engine then WAL is the part of transactions that make it work properly. Without them it is impossible to know what to rollback.

This mechanism guarantees that:

- Every update has a corresponding log entry.
- In case of a system failure, the database can reconstruct the correct state using the log records.
- Incomplete transactions can be safely rolled back during recovery.

→ Check Points:

What if a database is being used for years and has billions of logs. Clearly, if the system crashed, then performing all transactions in the logs file to recover the database back is **NIGHTMARE**.

It would take lot of time and is very inefficient.

So to solve this, as in many video games like BloodBorne, Lies of P etc. (in any game) we have checkpoints so that our progress doesn't lose, Similarly we have implemented checkpoint mechanism for our engine too.

After a threshold number of successful transactions, we store the whole current database in the systems_data.db disk file. And then we can delete all the logs as now we don't need them as we have stored current position of database.

```
# Automatic Checkpoint Logic
self.commit_count += 1
if self.commit_count >= self.checkpoint_threshold:
    print(f"\n--- Auto-Checkpoint: {self.commit_count} commits reached ---")
    self.checkpoint()
```

We added this in commit. As soon as successful transactions reach threshold, we save entire database to disk file

Hence, by this way the logs file don't grow crazily and whenever system crash, the database can be loaded from the disk file and all logs after checkpoint can be performed. By this way we can recover our database.

```
# Saving automatically on disk
def checkpoint(self):
    # Syncs the B+ Tree to system_data.db and clears the WAL.
    self.db.save_to_disk() # Long-term storage
    self.wal.reset()       # Clear short-term log
    self.commit_count = 0  # Reset counter
    print("System synced. WAL truncated.\n")
```

So in short, Transaction Manager, WAL, Checkpoints, B+ Tree and logical constraints we added handle the correctness of all operations and ensure that database is in valid state always as transaction starts and as it ends.

Hence, through the combination of **B+ Tree indexing**, **transaction management**, and **write-ahead logging**, the system ensures that all operations are executed correctly and that the integrity of the database is maintained at all times.

🌀 How failures are handled:

While correctness ensures the database follows the rules during normal operation, failure handling ensures the database survives when anything that could go wrong, goes wrong. So in our database engine, we dealt with two cases of failure:

1. Soft Failures: Transaction Rollbacks:

A soft failure occurs when one of the operation in a transaction gets interrupted due to logical error (duplicate primary key or wrong update) or due to some technical error.

So to handle such errors, we use the Transaction rollback mechanism:

We wrap our transaction logic in a try except block and if the exception is caught, then in the except block we have written:

```
tm.rollback()
```

When this command is triggered, it reads the current transaction's operations from the log entries in reverse order and performs the Undo operation.

To simulate this reverse behavior, we stored the operations of a transaction in a stack like data structure. Hence as we perform operation, we push in stack and during rollback, we pop from it and then perform the undo operation using operations.py module.

```
# Rollback
def rollback(self, tid):
    if tid not in self.operation_stack:
        print("Transaction not active")
        return

    print(f"Rolling back transaction {tid}")

    stack = self.operation_stack[tid]

    while stack:
        op = stack.pop()
        op.undo()

    del self.operation_stack[tid] # rollback so removing it
```

Hence, that's how we can rollback.

Result: The database is returned to its state as of the last tm.begin(), ensuring Atomicity.

2. Hard Failures: System Crashes and Recovery:

So a hardware failure is a complete loss of RAM due to power outage or a system crash. Since, B+ Tree lives inside the RAM, it is completely wiped out during such an event.

So to help us to recover from such dangerous situations, we have **Checkpoint + WAL** mechanism. So whenever a system undergoes crash, then since checkpoint stores the entire database in disk file, we can load the entire database from the file after the system resets. Then we can get the database that was stored at the last checkpoint.

And then to get to the latest database before crash, we would have logs stored after the checkpoint. So we can perform the operations in log and hence by that way we can get our database back to its last committed state. So the process is like:

Upon restarting, the system follows a two-step recovery:

1. **Reloading the Checkpoint:** The db.load_from_disk() function loads the last consistent state from the system_data.db file. Hence we have database stored at last checkpoint.
2. **Replaying the WAL:** The wal.recover() function scans the wal.log. It identifies any transaction that has a COMMIT tag but whose changes aren't in the checkpoint yet. It Redoes these operations on the B+ Tree and hence bring the database back to its original state.

```

# Now begins the final test: Durability test
print("\n--- Durability Test ---")

# Starting transaction
t1 = tm.begin()
student.update(230005, {"CPI": 9.9}, tm, t1)
course.update(201, {"LTPC": "3_2_2_5"}, tm, t1)
enrollment.insert((8, 230020, 101, "2023"), tm, t1)

# Committing transaction
tm.commit(t1)

# _____ CRASH _____ #
print("SYSTEM CRASH!!!!!!")

# Simulating crash: reinitializing the database to restart the system.
db = DB_Manager()
db.load_from_disk() # Load schema + data from last checkpoint
db.use_database("Attendance_management_portal")

# Now firstly lets see content in wal.Log
wal_log = WAL()
print("\nWrite Ahead Log Contents")
print("-----")
wal_log.show_logs()
print("-----")

print("Now we can use this to get back and recover the Committed data!")
wal_log.recover(db)

```

Hence by this way we can perform the full recovery after system crash.

So that's how our system handles the failure and recovers.

🌀 How multi-user conflicts are handled:

In any real database, many users try to perform transactions at a same time in a database. It is also possible they may try to attempt to get same data simultaneously. To prevent “Dirty Reads” – where one user sees the another user’s uncommitted data and potentially incorrect data – our engine implements a **Snapshot – Isolation Strategy**:

- **The Mechanism:** Each transaction operates within its own isolated context. Changes made during a transaction are staged in the Write-Ahead Log and the transaction's private buffer. This means that the operations of a transaction would seem to have performed but won't modify the B+ Tree until and unless it is committed.
- **The Guardrail:** The B+ Tree index only returns data that has been officially Committed. If User A is currently updating a student's CPI but has not yet finished, User B will continue to see the old, stable value from the B+ Tree.
- **Result:** This ensures that every user has a consistent and "truthful" view of the database, regardless of what other background transactions are doing.

So to perform this, what we did is:

1. We make sure that until and unless `tm.commit()` is called, the B+ Tree should not update our database. Hence, whenever a user performs transactions, it sees only the old values.
2. Once the transaction is committed, then logs are written and then only B+ Tree modifies the database.

```

# Commit
def commit(self, tid):
    if tid not in self.operation_stack:
        print("Transaction not active")
        return

    # We log commit first before performing operations. Write ahead logging
    self.wal.log_commit(tid)

    # now we can apply staged operations now
    for op in self.operation_stack[tid]:
        op.perform()

```

Hence, all the updates: Insert, delete and update modify the database once the user commits the database. Other users accessing the same database would see the old values only until transaction is committed.

Hence, by this way a user can't see another user's uncommitted data and hence preventing dirty reads. So the transactions would work as if they are happening sequentially. Until the second user commits his transaction, he won't be able to see the uncommitted transaction that first user is performing.

This is Isolation and hence our database handles it very well!

☞ What experiments were performed:

To verify the correctness and robustness of our Mini Database Engine, we performed several tests. These tests were designed to validate transaction correctness, rollback behavior, crash recovery, and isolation between concurrent users.

We distributed the tests such that each test focuses on one of the core properties: ACID. So our mini database under went mostly 4 main tests. So let's see the tests:

1. Atomicity Test:

In atomicity, we performed two major tests:

α. Successful Test:

So in this test, we performed a transaction:

```
print("\n--- Atomicity Test: Successful Transaction ---")

tid = tm.begin()

# Performing transaction
print("Operation 1 → Insert Student")
student.insert((230008,"23PH001",2023,7.9), tm, tid) # adding a new student
print("Operation 2 → Update Course")
course.update(101, {"LTPC":"3_1_0_5"}, tm, tid) # updating the course DSA to 5 credits
print("Operation 3 → Insert Enrollment")
enrollment.insert((7,230008,101,"2023"), tm, tid) # adding the new child to the enrollment table

# Transaction performed, now committing
tm.commit(tid)
```

And since there was no failure, as expected, our B+ Tree successfully performed the desired changes. Hence in final table:

- Entry (230008,"23PH001",2023,7.9) got added in student table
- The course id 101 got updated its LTPC to 3_1_0_5
- Entry (7,230008,101,"2023") got inserted in enrollment table:

After transaction:Database State

```
Table: student
['Student_id', 'Roll_No', 'Batch_Year', 'CPI']
(230005, '23CS001', 2023, 8.4)
(230008, '23PH001', 2023, 7.9)
(230011, '23CS002', 2023, 9.1)
(240001, '24CS001', 2024, 8.8)
(250003, '25CS001', 2025, 8.9)

Table: course
['Course_id', 'Course_code', 'Course_Name', 'LTPC']
(101, 'CSE101', 'Data_Structures_and_Algorithms', '3_1_0_5')
(102, 'CSE102', 'Database_Management', '3_0_0_4')
(103, 'CSE103', 'Cybersecurity', '3_0_1_3')
(201, 'AI101', 'Deep_Learning', '3_0_0_4')

Table: enrollment
['Enrollment_id', 'Student_id', 'Course_id', 'Academic_Year']
(1, 230005, 101, '2024')
(2, 230005, 102, '2024')
(3, 230011, 101, '2024')
(4, 240001, 103, '2025')
(5, 250003, 201, '2025')
(6, 230011, 102, '2024')
(7, 230008, 101, '2023')
```

Hence, our engine successfully committed ALL operations in transaction.

(For better results comparison, our GitHub link has the notebook as well as html file. You can understand better by comparing initial and final table)

β. Failure Test:

So in this test, we performed transaction:


```

tid = tm.begin()

# Performing transaction
print("Operation 1 → Insert Student")
student.insert((230014,"23AI001",2023,8.2), tm, tid) # adding a new student
print("Operation 2 → Update Course")
course.update(103, {"LTPC":"3_0_0_4"}, tm, tid) # updating the course Cybersecurity to 4 credits and removing Lab slot
print("Operation 3 → Insert Enrollment")
enrollment.insert((8,230014,101,"2023"), tm, tid) # adding the new child to the enrollment table

# Simulating failure
print("Simulating failure...")
print("CRASH!!!!!!!!!!!!!!!!!!!!!!")
tm.rollback(tid)

```

So here to simulate failure, instead of commit(), we let there be some technical error due to which transaction failed at third operation and hence we called rollback:

```

--- Atomicity Test: Failed Transaction ---
Transaction 5 started
Operation 1 → Insert Student
Operation 2 → Update Course
Operation 3 → Insert Enrollment
Simulating failure...
CRASH!!!!!!!!!!!!!!!!!!!!!!
Rolling back transaction 5

```

And as expected our database undo all the three operations and hence our database didn't get modified:

```

Table: student
['Student_id', 'Roll_No', 'Batch_Year', 'CPI']
(230005, '23CS001', 2023, 8.4)
(230008, '23PH001', 2023, 7.9)
(230011, '23CS002', 2023, 9.1)
(240001, '24CS001', 2024, 8.8)
(250003, '25CS001', 2025, 8.9)

Table: course
['Course_id', 'Course_code', 'Course_Name', 'LTPC']
(101, 'CSE101', 'Data_Structures_and_Algorithms', '3_1_0_5')
(102, 'CSE102', 'Database_Management', '3_0_0_4')
(103, 'CSE103', 'Cybersecurity', '3_0_1_3')
(201, 'AI101', 'Deep_Learning', '3_0_0_4')

Table: enrollment
['Enrollment_id', 'Student_id', 'Course_id', 'Academic_Year']
(1, 230005, 101, '2024')
(2, 230005, 102, '2024')
(3, 230011, 101, '2024')
(4, 240001, 103, '2025')
(5, 250003, 201, '2025')
(6, 230011, 102, '2024')
(7, 230008, 101, '2023')

```

If we see, it is same as the one we had in atomicity one.

This shows that if there is failure in even one operation of a transaction, **NOTHING** will happen in database.

So that's how our database performs ALL or NOTHING → Atomicity.

2. Consistency Test:

So consistency means that a transaction should always commit by leaving database in valid state, i.e., a database must never be left in invalid state after transaction ends.

So to demonstrate the consistency, we focused on two major areas that each table must have:

α. Primary key Uniqueness:

So here, we forcefully tried to perform a transaction where a primary key already existing is being added, i.e.,

```

tid = tm.begin()

try:
    print("Operation 1 → Insert Student")
    student.insert((230020,"23AI010",2023,8.2), tm, tid)

    print("Operation 2 → Update Course")
    course.update(101, {"LTPC":"3_0_0_4"}, tm, tid)

    print("Operation 3 → Insert Enrollment with duplicate key")
    enrollment.insert((1,230020,101,"2023"), tm, tid) # 1 already exists --> ERROR as primary key being tried to be duplicated

    tm.commit(tid)

except Exception as e:
    print("Consistency violation detected:", e)
    tm.rollback(tid)

```


Hence, here enrollment id already had 1 as its primary key. But we forcefully added it again. And if we see output:

```
--- Consistency Test ---
Transaction 6 started
Operation 1 → Insert Student
Operation 2 → Update Course
Operation 3 → Insert Enrollment with duplicate key
Consistency violation detected: Duplicate primary key
Rolling back transaction 6
```

Clearly, it lead to ValueError and hence resulted in rollback the transaction 6 maintaining consistency and atomicity at the same time. This shows:

“Consistency is Goal and Atomicity is tool!”

Hence, our database successfully handles the uniqueness of primary key and ensure database doesn't enter invalid state with duplicate primary keys.

β. No Wrong Updates:

Though not a major part of consistency, but still the transaction must rollback if it finds error such that an entry is being tried to get updated even when it doesn't exist. So considering it part of consistency, our another test was to forcefully do a update to an entry that doesn't exist.

```
# Beginning the transaction
>print("\n--- Consistency Test ---")

tid = tm.begin()

try:
    print("Operation 1 → Insert Student")
    student.insert((230020,"23AI010",2023,8.2), tm, tid)

    print("Operation 2 → Update Course")
    course.update(110, {"LTPC":"3_0_0_4"}, tm, tid) # Primary key 110 doesn't exist

    print("Operation 3 → Insert Enrollment")
    enrollment.insert((1,230020,101,"2023"), tm, tid) # 1 already exists --> ERROR as primary key being tried to be duplicated

    tm.commit(tid)

except Exception as e:
    print("Consistency violation detected:", e)
    tm.rollback(tid)
```

Hence we tried to update data at key 110 which don't even exist. Here we also had duplicated primary key. But since it saw the there was error before it, it immediately called off the transaction and rollback. This we can see in output:

```
--- Consistency Test ---
Transaction 7 started
Operation 1 → Insert Student
Operation 2 → Update Course
Consistency violation detected: Entry Doesn't exist
Rolling back transaction 7
```

Hence, our system correctly identifies wrong updates and immediately rollbacks before they can bring database in an invalid state.

By this way, our database passes the Consistency test too.

3. Isolation Test:

The main objective of ours in isolation test was not to allow dirty reads, i.e., uncommitted data of one user must not be visible to another user who is performing operation on same data.

For example:

User A modifies the CPI of student C from 8.7 to 9.1. While User B wants to see the CPI of same student C. Then the B must see 8.7 (old one) and NOT (9.1) as that's uncommitted data. It means one transaction is affecting another transaction which must never happen. This is dirty read.

So with focus that multiple transactions must happen such that it seems like they are happening one by one, we performed the Isolation test:

```
# Now begins the third test: Isolation test
print("\n--- Isolation Test ---")

t1 = tm.begin()
student.update(230005, {"CPI": 9.5}, tm, t1)

print("T1 updated CPI but not committed yet")

t2 = tm.begin()

print("T2 reads student record:")
print(student.search(230005))

tm.commit(t1)
tm.commit(t2)
```

So here, to demonstrate the multiple transactions at a time, we didn't commit t1 and continued t2. This shows multiple users are working at same time.

Now, T1 updates the CPI of student 230005 to 9.5 (from 8.4) but T2 reads the CPI of same student. To avoid dirty reads, the T2 should print 8.4 rather 9.5 until both transactions commit.

Once they are committed then 9.5 should be visible to all.

So if we see output:

```
--- Isolation Test ---
Transaction 8 started
T1 updated CPI but not committed yet
Transaction 9 started
T2 reads student record:
(230005, '23CS001', 2023, 8.4)
Transaction 8 committed
Transaction 9 committed
```

Clearly, the T2 reads CPI of student as 8.4 and hence this ensures there is no dirty read.

Note: Here we used only one table, because isolation is a complex test to demonstrate. So for simplicity to understand the mechanics of it, we used only one table with only one entry example.

This is easy to verify and demonstrates very clearly.

Hence our database correctly removes the dirty reads implementing Isolation.

4. Durability Test:

The final and the most important property of a database is Durability, i.e., even after a system crash it should be able to recover its database. And as discussed before, Checkpoints and the WAL logs help us to recover the whole database.

So the test we performed is:

```

# Starting transaction
t1 = tm.begin()
student.update(230005, {"CPI": 9.9}, tm, t1)
course.update(201, {"LTPC": "3_2_2_5"}, tm, t1)
enrollment.insert((8, 230020, 101, "2023"), tm, t1)

# Committing transaction
tm.commit(t1)

# _____ CRASH _____ #
print("SYSTEM CRASH!!!!!!!")

# Simulating crash: reinitializing the database to restart the system.
db = DB_Manager()
db.load_from_disk() # Load schema + data from last checkpoint
db.use_database("Attendance_management_portal")

# Now firstly lets see content in wal.Log
wal_log = WAL()
print("\nWrite Ahead Log Contents")
print("-----")
wal_log.show_logs()
print("-----")

print("Now we can use this to get back and recover the Committed data!")
wal_log.recover(db)

# Verify recovery
print("After recovery:", student.search(230005))
# Expected: CPI = 9.9 (committed change survives crash)

```

Hence, to simulate that our system failure, we reinitialized the whole database again. This create a new database from absolute scratch. No B+ Tree inside it, no tables, no database we created. An empty Void!

Then with the help of checkpoint, we brought the whole database back to the db object we created and this brought the database we created back into it. Then with help of logs, we performed all the operations in it with help of wal.recover(). And by this way our database returned to the original state. And then we can check the student's CPI which we updated to 9.9.

So if we see result:

```

--- Durability Test ---
Transaction 10 started
Transaction 10 committed
SYSTEM CRASH!!!!!!!
Database state loaded from disk.
Using database Attendance_management_portal

Write Ahead Log Contents
-----
Begin 10
10|student|update|230005|(230005, '23CS001', 2023, 9.5)|(230005, '23CS001', 2023, 9.9)
10|course|update|201|(201, 'AI101', 'Deep_Learning', '3_0_0_4')|(201, 'AI101', 'Deep_Learning', '3_2_2_5')
10|enrollment|insert|8|None|(8, 230020, 101, '2023')
COMMIT 10
-----

Now we can use this to get back and recover the Committed data!
Redoing committed transaction 10 to ensure durability.
After recovery: (230005, '23CS001', 2023, 9.9)

```

Clearly, here we got our database back after crash and the latest transaction we performed is still in the database. This shows that our database survived the crash and hence recovered to its last committed state.

That's durability and our database engine passes that with full colors!

🌀 Observations and limitations:

➔ Observations:

From the experiments performed on our Mini Database Engine, several key observations were made:

1. Transactions behave correctly under both success and failure conditions.

When all operations in a transaction execute successfully, the system commits them together. However, if any

operation fails, the rollback mechanism correctly restores the database to its previous state. This explains the atomicity.

2. The Write-Ahead Logging (WAL) mechanism plays a crucial role in maintaining reliability.

By recording operations before they are applied, the system is able to undo failed transactions and redo committed ones during recovery. This is the most important part of our database as it helps in rollback and works with checkpoint to bring database back during system crash.

3. Check pointing significantly improves crash recovery efficiency.

Instead of replaying the entire log from the beginning, the system starts recovery from the latest checkpoint and then applies the WAL entries. This reduces the recovery time. A good game needs checkpoint and hence similarly, even a good database needs that too otherwise both are very annoying.

4. Isolation prevents dirty reads between concurrent transactions.

Even when two transactions access the same data simultaneously, uncommitted changes remain invisible to other transactions until a commit occurs.

These observations demonstrate that the implemented system successfully maintains the **ACID properties** required for a reliable database engine.

→ **Limitations:**

While our database system engine demonstrates the core principles of ACID properties, there are several limitations in the current implementation that we even talked about previously:

1. Limited concurrency control.

The system simulates multi-user behavior but does not implement advanced concurrency control mechanisms such as locking protocols used in real-world databases. Hence it has no security. If someone itself deletes the logs before checkpoint, our database system unfortunately becomes helpless. Even the disk file systems_data.db has no security.

2. Generic table design.

The table implementation is intentionally kept generic, which makes it difficult to enforce table-specific constraints such as domain validation (e.g., CPI ranges or roll number formats). This was one of the main drawbacks of our B+ Tree engine.

It can't pass real concurrency test like age>0, CPI>0, name cannot be number etc. This is because tables we implemented are very simple and generic. If we tried to implement any one of such constraint, then it would lose its genericity and hence we won't be able to use same code for rest tables. This would make it very complex and hence to keep it simple, we avoided that and focused on concurrency tests that every table must have in common like primary key uniqueness and no wrong updates.

3. Simplified recovery mechanism.

The recovery process assumes a simplified logging structure and does not implement advanced techniques such as log truncation or complex crash scenarios.

4. Memory-based B+ Tree structure.

Since the B+ Tree primarily resides in memory, the system relies heavily on checkpoints and logs for persistence. A fully disk-based index structure would make the system more robust. So our engine uses lots of memory for disk file, log file and even B+ Tree. Hence it's not memory optimized.

Despite these limitations, the implemented system successfully demonstrates the fundamental concepts of **transaction management, failure recovery, and ACID compliance** and hence is the best way to demonstrate these properties through our toy database.



Module – B: Multi-User Behaviour and Stress Testing

The GitHub Link to access the codebase is given below:

https://github.com/AryanMeshram059/db_3_module_b

The link to the YouTube video containing explanation of this module:

<https://drive.google.com/file/d/17KU57JHNKixcGwtAtetkh7p3PtLxAfeS/view>

🌀 Introduction:

Modern web-bases systems operate in highly concurrent environments where multiple users interact with shared resources simultaneously. In such systems, ensuring **correctness**, **consistency**, and **reliability of data** becomes a critical challenge. If concurrent operations are not handled properly, **anomalies such as race conditions, lost updates, or partial data** writes may occur.

While Module A focused on implementing the internal mechanisms of a reliable database engine, including transactions and recovery, Module B evaluates how the application behaves when accessed by multiple users through the API layer.

The main objective of this module are:

- To simulate **concurrent user activity** and analyse system behaviour.
- To identify **potential race conditions** in request processing.
- To resolve **concurrency issues** using atomic database operations.
- To ensure reliability through **transaction-based failure handling**.
- To evaluate system behaviour under high load through **stress testing**.

These experiments demonstrate that the system maintains data integrity and consistent behaviour even when multiple users interact with it simultaneously.

➔ Concurrent Usage Simulation:

In real-world systems, multiple users may perform operations on the database at the same time. Since the backend of the attendance management system is implemented using **Node.js and Express**, the server processes HTTP requests asynchronously. This allows the application to handle multiple incoming requests concurrently.

To simulate this behaviour, a testing script was developed using Node.js. The script generates multiple API requests in parallel to imitate different administrators interacting with the system simultaneously.

The concurrency experiment targeted the API endpoint responsible for processing attendance requests:

```
PUT /api/request/:id
```

Each request contains an administrator authentication token and attempts to update the status of the same request record.

The testing script repeatedly sends requests with alternating status values (Approved and Rejected), thereby simulating a realistic scenario where multiple administrators attempt to process the same request at nearly the same time.

```

for (let i = 0; i < 8; i++) {
  fetch(URL, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `${tokens[i % tokens.length]}`
    },
    body: JSON.stringify({
      status: i % 2 === 0 ? "approved" : "rejected"
    })
  })
  .then(res => res.text())
  .then(data => console.log(`Admin ${i}:`, data));
}

```

Example output snapshot:

```

[2026-04-04T17:46:10.671Z] UPDATE_REQUEST_SUCCESS: admin=9001 requestId=1 status=approved
[2026-04-04T17:46:10.677Z] UPDATE_REQUEST_SUCCESS: admin=9003 requestId=1 status=approved
[2026-04-04T17:46:10.676Z] UPDATE_REQUEST_SUCCESS: admin=9002 requestId=1 status=rejected
[2026-04-04T17:46:10.677Z] UPDATE_REQUEST_SUCCESS: admin=9004 requestId=1 status=rejected
[2026-04-04T17:46:10.679Z] UPDATE_REQUEST_SUCCESS: admin=9001 requestId=1 status=approved
[2026-04-04T17:46:10.680Z] UPDATE_REQUEST_SUCCESS: admin=9002 requestId=1 status=rejected
[2026-04-04T17:46:10.681Z] UPDATE_REQUEST_SUCCESS: admin=9004 requestId=1 status=rejected
[2026-04-04T17:46:10.680Z] UPDATE_REQUEST_SUCCESS: admin=9003 requestId=1 status=approved

```

Server log showing concurrent admin updates

→Race Condition Analysis:

A race condition occurs when multiple concurrent operations attempt to modify the same shared resource and the final outcome depends on the order in which the operations are executed.

In the attendance management system, a race condition may occur when multiple administrators attempt to process the same attendance request simultaneously.

For example:

- Admin A sends a request to approve a request.
- Admin B sends a request to reject the same request.

If both updates are executed independently without synchronization, the following SQL queries may run concurrently:

```

UPDATE attendance_request
SET Status = 'Approved'
WHERE RequestID = 60;

UPDATE attendance_request
SET Status = 'Rejected'
WHERE RequestID = 60;

```

Since these queries may execute at nearly the same time, the final state of the record depends on which query finishes last. This leads to unpredictable behaviour where one update overwrites another. Such a scenario is commonly referred to as the **lost update problem**.

Race conditions violate important database guarantees such as consistency and isolation, and therefore must be prevented.

→Race Condition Resolution Using Atomic Updates:

To eliminate the race condition, a conditional atomic update mechanism was implemented at the database level.

Instead of performing a simple update, the SQL query was modified to update the record only if the request status is still **Pending**.

```
UPDATE attendance_request
SET Status = ?, ProcessedBy = ?
WHERE RequestID = ? AND Status = 'Pending';
```

This query ensures that the update operation succeeds only if the request has not yet been processed.

The execution flow works as follows:

1. Multiple administrators may send update requests simultaneously.
2. The database checks the condition, Status = 'Pending'.
3. The first request successfully updates the record.
4. Once the status changes, subsequent requests fail the condition.
5. The server returns a message indicating that the request has already been processed.

This approach ensures that only one administrator can successfully process a request while all other concurrent attempts are rejected.

Code Snippet: Backend Route Handling

```
router.put("/request/:id", auth, allowRoles("Admin"), (req, res) => {
  const { status } = req.body;
  const requestId = req.params.id;

  db.query(
    `UPDATE attendance_request
     SET Status = ?, ProcessedBy = ?
     WHERE RequestID = ? AND Status = 'pending`,
    [status, req.user.id, requestId],
    (err, result) => {
      if (err) return res.status(500).send("Database error ✖");

      if (result.affectedRows === 0) {
        return res.send("Already processed by another admin ✖");
      }

      res.send("Request updated safely ✔");
    }
  );
});
```

Terminal Output:


```
Admin 1: Request updated safely ✓
Admin 2: Already processed by another admin ✗
Admin 0: Already processed by another admin ✗
Admin 4: Already processed by another admin ✗
Admin 3: Already processed by another admin ✗
Admin 5: Already processed by another admin ✗
Admin 6: Already processed by another admin ✗
Admin 7: Already processed by another admin ✗
```

→ Transaction-Based Failure Handling:

In practical applications, database operations may fail due to unexpected errors, network interruptions, or system failures. If a failure occurs in the middle of an operation, partial data may be written to the database, leading to inconsistency.

To prevent this issue, request creation was implemented using **database transactions**.

The transaction workflow is as follows:

1. A transaction is started using `beginTransaction`.
2. A new request record is inserted into the database.
3. If a failure condition occurs, the transaction is rolled back.
4. Otherwise, the transaction is committed.

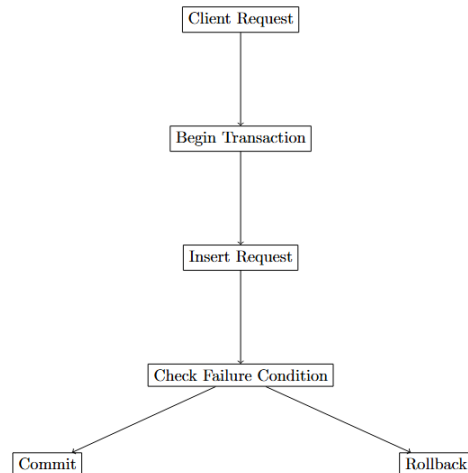
Code Snippet: Transaction Logic

```
db.query(
  `INSERT INTO attendance_request
  (AttendanceID, StudentID, Reason, Status)
  VALUES (?, ?, ?, 'Pending')`,
  [attendanceId, studentId, reason],
  (err, result) => {

    if (reason.includes("FAIL")) {
      return db.rollback(() => {
        res.status(500).json({
          message: "Simulated failure → rollback executed"
        });
      });
    }

    db.commit((err) => {
      res.json({
        message: "Request created",
        requestId: result.insertId
      });
    });
  }
);
```

Transaction Flow:



Terminal Output:

```
Request 1 (Student 240004, Attendance 5): { message: 'Request created ✓', requestId: 61 }
Request 2 (Student 250003, Attendance 16): { message: 'Request created ✓', requestId: 62 }
Request 3 (Student 230008, Attendance 32): { message: 'Request created ✓', requestId: 63 }
Request 5 (Student 240004, Attendance 74): { message: 'Request created ✓', requestId: 64 }
Request 7 (Student 230008, Attendance 88): { message: 'Request created ✓', requestId: 65 }
Request 6 (Student 250003, Attendance 79): { message: 'Request created ✓', requestId: 66 }
Request 9 (Student 240004, Attendance 4): { message: 'Request created ✓', requestId: 67 }
Request 10 (Student 250003, Attendance 5): { message: 'Simulated failure → rollback executed ✗' }
Request 11 (Student 230008, Attendance 16): { message: 'Simulated failure → rollback executed ✗' }
Request 13 (Student 240004, Attendance 45): { message: 'Simulated failure → rollback executed ✗' }
Request 14 (Student 250003, Attendance 74): { message: 'Simulated failure → rollback executed ✗' }
```

→ Stress Testing:

To evaluate the behaviour of the system under concurrent load, stress testing was conducted by generating multiple request creation operations through an automated script. Stress testing is used to assess the stability and reliability of a system when it receives a large number of requests within a short time period.

A testing script was developed using Node.js (testFailure.js) to simulate concurrent student requests to the API endpoint responsible for creating attendance requests.

The script repeatedly sends HTTP POST requests to:

```
POST /api/request
```

Each request is sent using a valid student authentication token and includes parameters such as `studentId`, `attendanceId`, and `reason`. Some requests intentionally include the keyword **"FAIL"** in the `reason` field to trigger a simulated failure condition and test the rollback mechanism implemented using transactions.

Experimental Setup:

In this experiment, **100 concurrent request creation operations** were generated using the testing script. The requests were distributed across multiple student accounts and attendance records to simulate realistic system usage. A portion of the requests intentionally triggered simulated failures to verify the correct functioning of the transaction rollback mechanism.

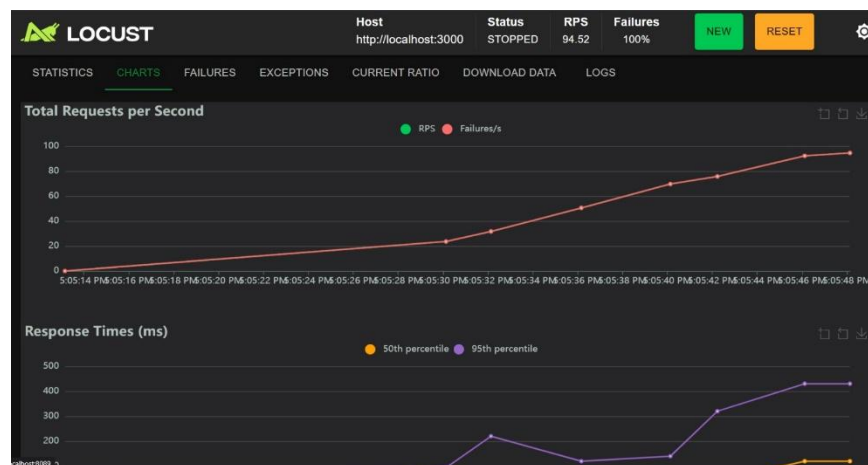
In addition to the custom testing script, load testing was also performed using the tool **Locust**, which allows simulation of multiple virtual users interacting with the backend server simultaneously. Locust provides real-time performance metrics such as request throughput, response time distribution, and failure rate.

Observations:

The following observations were recorded during the stress testing experiment:

- The Node.js backend successfully handled multiple concurrent requests without crashing.
- Requests containing the keyword **FAIL** correctly triggered transaction rollbacks.
- No partial or inconsistent records were inserted into the database.
- Requests without failure conditions were successfully committed.

Output Snapshot:



Locust Stress Testing Result:

The Locust dashboard provided performance metrics such as requests per second (RPS), response time percentiles, and failure rate. During the experiment, the system achieved a peak throughput of approximately 94 requests per second, indicating that the backend was able to process a substantial number of API calls within a short time interval.

The response time distribution generated by Locust showed the following approximate values:

- 50th percentile response time: around 120 ms
- 95th percentile response time: around 430 ms

These metrics indicate that the majority of requests were processed quickly, while a smaller percentage experienced higher response times as the request load increased.

The Locust output also displayed a high failure percentage. This behaviour was expected because several requests intentionally triggered simulated failure conditions within the application logic. In particular, requests containing the keyword "FAIL" activate the rollback mechanism implemented in the request creation API. As a result, these requests are recorded as failures by the load testing tool even though the behaviour confirms the correct functioning of the transaction rollback process.

Analysis

The stress testing results demonstrate that the system is capable of handling a significant number of concurrent request operations while maintaining database consistency. The transaction mechanism ensures that database operations remain atomic, meaning that either the entire request is successfully stored or the operation is completely rolled back in the event of a failure.

Even under increasing request rates, the system continued to process operations correctly without producing inconsistent or partially written records. The observed failure rate corresponds to intentionally triggered rollback scenarios rather than system instability.

Overall, the experiment confirms that the implemented concurrency control and transaction management mechanisms function correctly under concurrent workload conditions.

→ Key Technical Observations

The experiments conducted in this module highlight several important aspects of concurrent database system behaviour.

First, the asynchronous request handling model used by Node.js allows the server to process **multiple incoming requests concurrently**. While this improves system responsiveness, it also introduces the possibility of race conditions when multiple users attempt to modify the same database record simultaneously.

Second, the **race condition** observed during request processing demonstrates the importance of enforcing concurrency control mechanisms at the database level. By introducing a conditional atomic update query, the system ensures that only one administrator can successfully process a request, preventing lost updates.

Third, the use of transactions during request creation guarantees **atomic execution of database operations**. If any step fails during the process, the rollback mechanism ensures that no partial or inconsistent data is stored.

Finally, the **stress testing experiment** confirmed that the system remains stable when handling multiple concurrent operations. Even under concurrent access, the database maintained consistent and correct state transitions.

→ Conclusion

Module B evaluated the behaviour of the attendance management system under **concurrent access and simulated failure conditions**. Through controlled experiments, potential race conditions were identified in the request processing workflow.

The race condition issue was resolved by implementing a **conditional atomic update query**, ensuring that only one administrator can process a request while subsequent concurrent attempts are safely rejected. Additionally, request creation was implemented using database transactions, allowing the system to perform rollbacks when simulated failures occur.

Stress testing further validated that the system can handle multiple concurrent requests while maintaining database consistency. The experiments demonstrate that the system correctly enforces concurrency control and transactional integrity.

Overall, the implemented mechanisms ensure that the application behaves reliably in a multi-user environment while preserving the correctness of the database state.

